

# Tokenization & Truncation

**Pre-Work Module 01**

**ECBS5200 — Practical Deep Learning Engineering**

# The problem

Deep learning models don't read text.

They read **numbers**.

So we need a way to turn text into numbers — reliably, reversibly, and efficiently.

That process is called **tokenization**.

## What a tokenizer does

Input: "I have a credit card billing problem"

Output: [50281, 42, 452, 247, 3168, 3120, 20047, 1895, 50282]

Each number is a **token ID** — an index into a vocabulary.

The vocabulary is fixed when the model is trained. You don't get to change it.

## Not word-level: subword tokenization

Most modern tokenizers split text into **subword pieces**, not whole words.

```
"overpayment" → ["over", "payment"]
```

```
"refinancing" → ["ref", "in", "ancing"]
```

```
"mortgage" → ["mort", "gage"]
```

### Why subwords?

- A word-level vocabulary would need millions of entries
- Subwords handle misspellings, rare words, and new words gracefully
- Fixed vocabulary size (ModernBERT: ~50k tokens)

# Special tokens

Every sequence gets **special tokens** added automatically:

```
[CLS] I have a credit card billing problem [SEP]
```

↓

```
[50281, 42, 452, 247, 3168, 3120, 20047, 1895, 50282]
```

- **[CLS]** (token 50281) — marks the start; its embedding becomes the "sentence representation"
- **[SEP]** (token 50282) — marks the end of the input

These are added by the tokenizer, not by you.

## Real example: a consumer complaint

Let's tokenize an actual complaint from our dataset:

```
"There are many mistakes appear in my report without my understanding."
```

After tokenization:

```
Tokens: ['[CLS]', 'There', 'are', 'many', 'mistakes', 'appear',  
         'in', 'my', 'report', 'without', 'my', 'understanding',  
         '.', '[SEP]']
```

```
Token IDs: [50281, 2512, 403, 1142, 16503, 3176, 275, 619,  
           1304, 1293, 619, 4685, 15, 50282]
```

```
Length: 14 tokens
```

That's a short one. Some complaints are much longer.

# The truncation problem

Models have a **maximum sequence length**. They can't process infinite text.

In this course, we'll use **max\_length = 128 tokens**.

What happens when a complaint is longer than 128 tokens?

```
"I have been dealing with this company for over six months  
regarding unauthorized charges on my account. First they  
told me to call back, then they said they would investigate,  
then they never followed up. I have documentation showing..."
```

```
[... tokens 1-128 kept, everything after: GONE]
```

**Truncation** = silently dropping the end of the input.

## Why 128?

| max_length | Complaints fully covered | Training time per epoch |
|------------|--------------------------|-------------------------|
| 64         | ~50%                     | ~18 min                 |
| <b>128</b> | <b>~99%</b>              | <b>~37 min</b>          |
| 256        | ~99.9%                   | ~74 min                 |
| 512        | ~100%                    | ~148 min                |

We pick 128 because it's the sweet spot:

- Covers virtually all complaints without truncation
- Training finishes in reasonable time
- Doubling to 256 buys almost nothing but **doubles** training time

This is a real engineering trade-off you'll encounter constantly.

# What the tokenizer actually returns

When you call a tokenizer in code, you get back a dictionary:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("answerdotai/ModernBERT-base")
result = tokenizer("I have a billing problem",
                  max_length=128,
                  truncation=True,
                  padding="max_length")
```

```
result.keys() → ['input_ids', 'attention_mask']
```

- **input\_ids** — the token ID sequence (the numbers the model reads)
- **attention\_mask** — 1 for real tokens, 0 for padding (ignore these)

## Padding: making sequences the same length

```
Complaint A (14 tokens): [50281, 2512, 403, ..., 15, 50282, 50283, 50283, ...]  
Attention mask:         [ 1, 1, 1, ..., 1, 1, 0, 0, ...]
```

```
Complaint B (128 tokens): [50281, 3245, 1107, ..., 2250, 50282]  
Attention mask:         [ 1, 1, 1, ..., 1, 1]
```

- Models process **batches** of sequences — all must be the same length
- Short sequences get **padded** with a special padding token
- The **attention mask** tells the model: "ignore the padding"

## Key takeaways

1. **Tokenizers turn text into numbers** — specifically, into sequences of subword token IDs
2. **The vocabulary is fixed** — it was set when the model was pretrained
3. **Special tokens** ([CLS], [SEP]) are added automatically
4. **Truncation silently drops text** beyond max\_length — your model never sees it
5. **Padding fills short sequences** so batches have uniform length
6. **max\_length is an engineering choice** — we chose 128 because it covers 99%+ of our data

**Next: try it yourself in the notebook!**